An Integrated Framework for the Development of CRDTs

A Deductive Study on Verification of Replicated Data Types

João Garção

Advisor: Mário Pereira

Co-Advisor: Carla Ferreira

November 20, 2026

Why Verification Matters

- Correctness Beyond Testing:
  - Tests check some executions, while verification proves a property for all of them.

Why Verification Matters

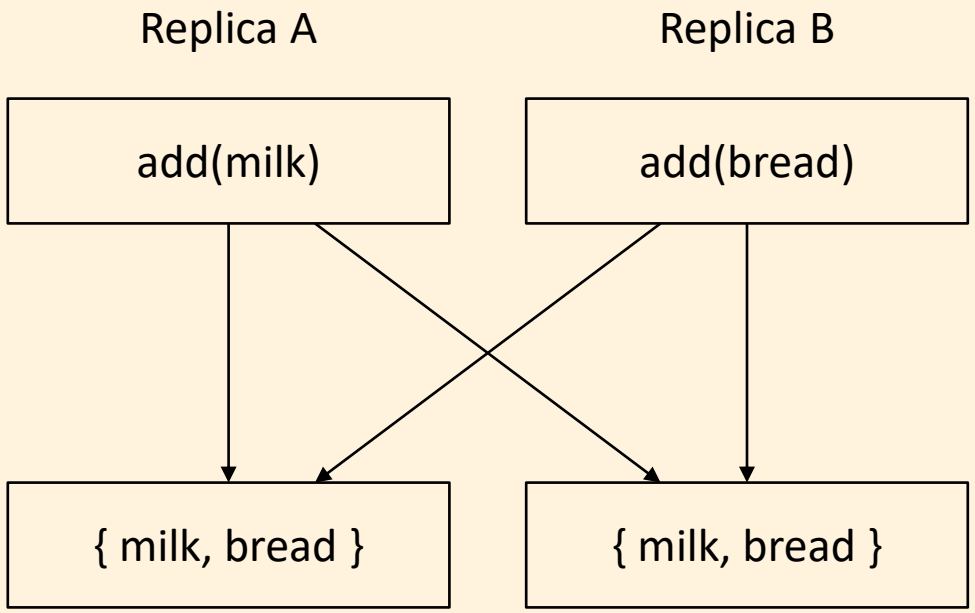
- Correctness Beyond Testing:
  - Tests check some executions, while verification proves a property for all of them.
- Code Generated by AI:
  - More and more code is written by AI tools, and it still has to be trusted.

Why Verification Matters

- Correctness Beyond Testing:
  - Tests check some executions, while verification proves a property for all of them.
- Code Generated by AI:
  - More and more code is written by AI tools, and it still has to be trusted.
- Distributed Systems:
  - Concurrent updates and message reordering lead to executions that are hard to test.

Replicated Data and CRDTs

- Replicas are updated independently, without coordination.
- CRDTs guarantee that all replicas converge to the same state.
- Implementing them correctly is still hard.



VeriFx

- Designed for CRDTs:
  - A Scala-like language to implement replicated data types and verify them.

VeriFx

- Designed for CRDTs:
  - A Scala-like language to implement replicated data types and verify them.
- Automatic Proofs:
  - The Z3 solver checks convergence and finds counterexamples, without help from the programmer.

VeriFx - Limitations

- Recursivity:
  - SMT solvers struggle with recursive definitions and unbounded structures, often leading to timeouts.

VeriFx - Limitations

- Recursivity:
  - SMT solvers struggle with recursive definitions and unbounded structures, often leading to timeouts.
- Semantics:
  - Proving convergence does not guarantee correctness, as the implementation can easily diverge from the programmer's original intention.

VeriFx - Limitations

- Recursivity:
  - SMT solvers struggle with recursive definitions and unbounded structures, often leading to timeouts.
- Semantics:
  - Proving convergence does not guarantee correctness, as the implementation can easily diverge from the programmer's original intention.
- Instability:
  - Minor, non-functional code changes can unpredictably break the automated verification process.

Why3

- Specification and Proof:
  - Programs in WhyML, with contracts, invariants and lemmas.
  - Proof obligations are sent to provers such as Alt-Ergo and Z3.

Why3

- Specification and Proof:
  - Programs in WhyML, with contracts, invariants and lemmas.
  - Proof obligations are sent to provers such as Alt-Ergo and Z3.
- Guided by the Programmer:
  - Lemmas and variants let induction be written instead of discovered.

Why3 - Limitations

- Manual Effort:
  - Lemmas, variants and assertions are written by hand.

Why3 - Limitations

- Manual Effort:
  - Lemmas, variants and assertions are written by hand.
- Its Own Language:
  - WhyML has its own syntax, ghost code and modelling choices.

Why3 - Limitations

- Manual Effort:
  - Lemmas, variants and assertions are written by hand.
- Its Own Language:
  - WhyML has its own syntax, ghost code and modelling choices.
- Two Separate Formalizations:
  - Using both tools means writing and maintaining every CRDT twice.

Research Questions

- Research Question 1:
  - Can a single specification language express both state-based and operation-based CRDTs, and be compiled automatically to VeriFx and Why3?

Research Questions

- Research Question 1:
  - Can a single specification language express both state-based and operation-based CRDTs, and be compiled automatically to VeriFx and Why3?
- Research Question 2:
  - Does writing a single specification reduce the effort of targeting both tools?

Research Questions

- Research Question 1:
  - Can a single specification language express both state-based and operation-based CRDTs, and be compiled automatically to VeriFx and Why3?
- Research Question 2:
  - Does writing a single specification reduce the effort of targeting both tools?
- Research Question 3:
  - Where do VeriFx and Why3 agree, and where does one go further than the other?

Background

Specification Language

Compiling to VeriFx and Why3

Case Studies and Evaluation

Conclusion

Future Work

Background - Synchronization Models

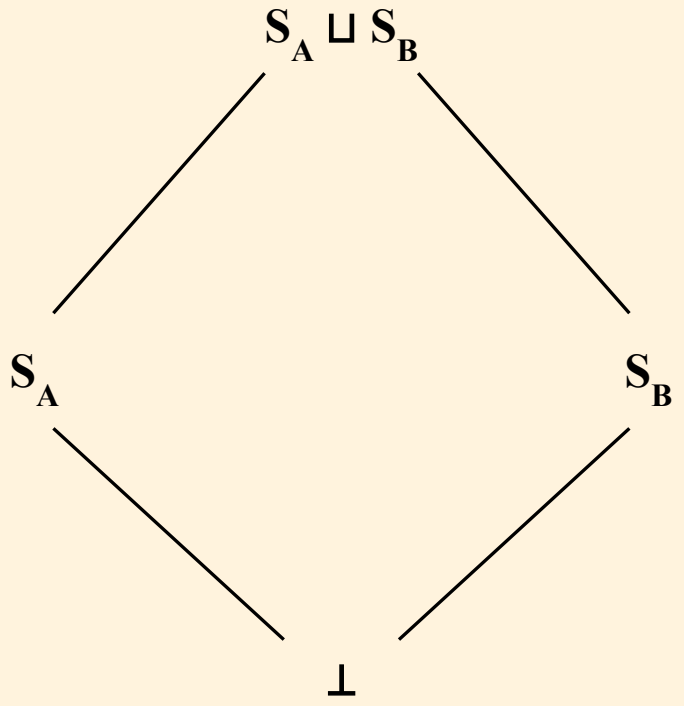
- Synchronization Models:
  - Convergent Replicated Data Types (CvRDTs)

Background - Synchronization Models

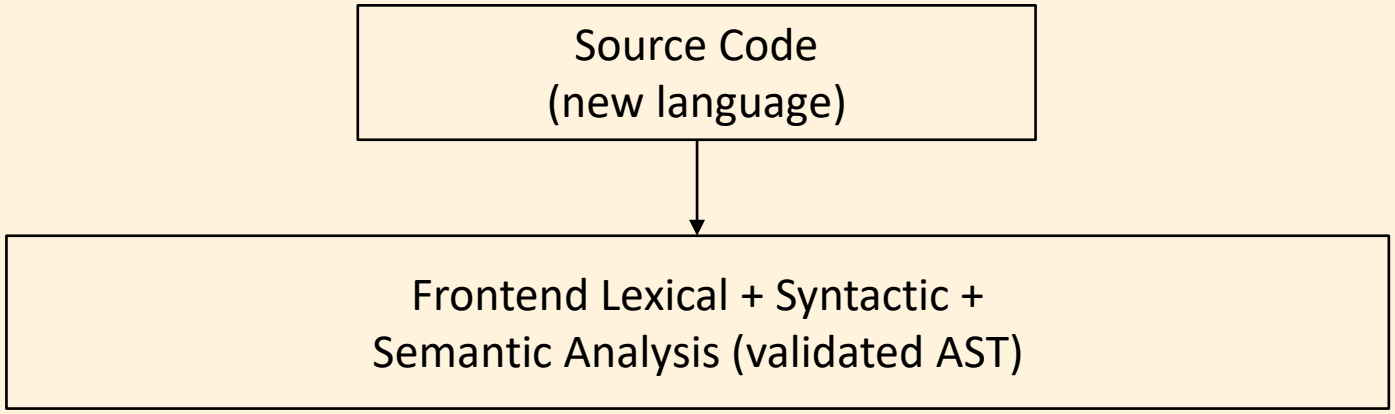
- Synchronization Models:
  - Convergent Replicated Data Types (CvRDTs)
  - Commutative Replicated Data Types (CmRDTs)

Background - Join-Semilattice

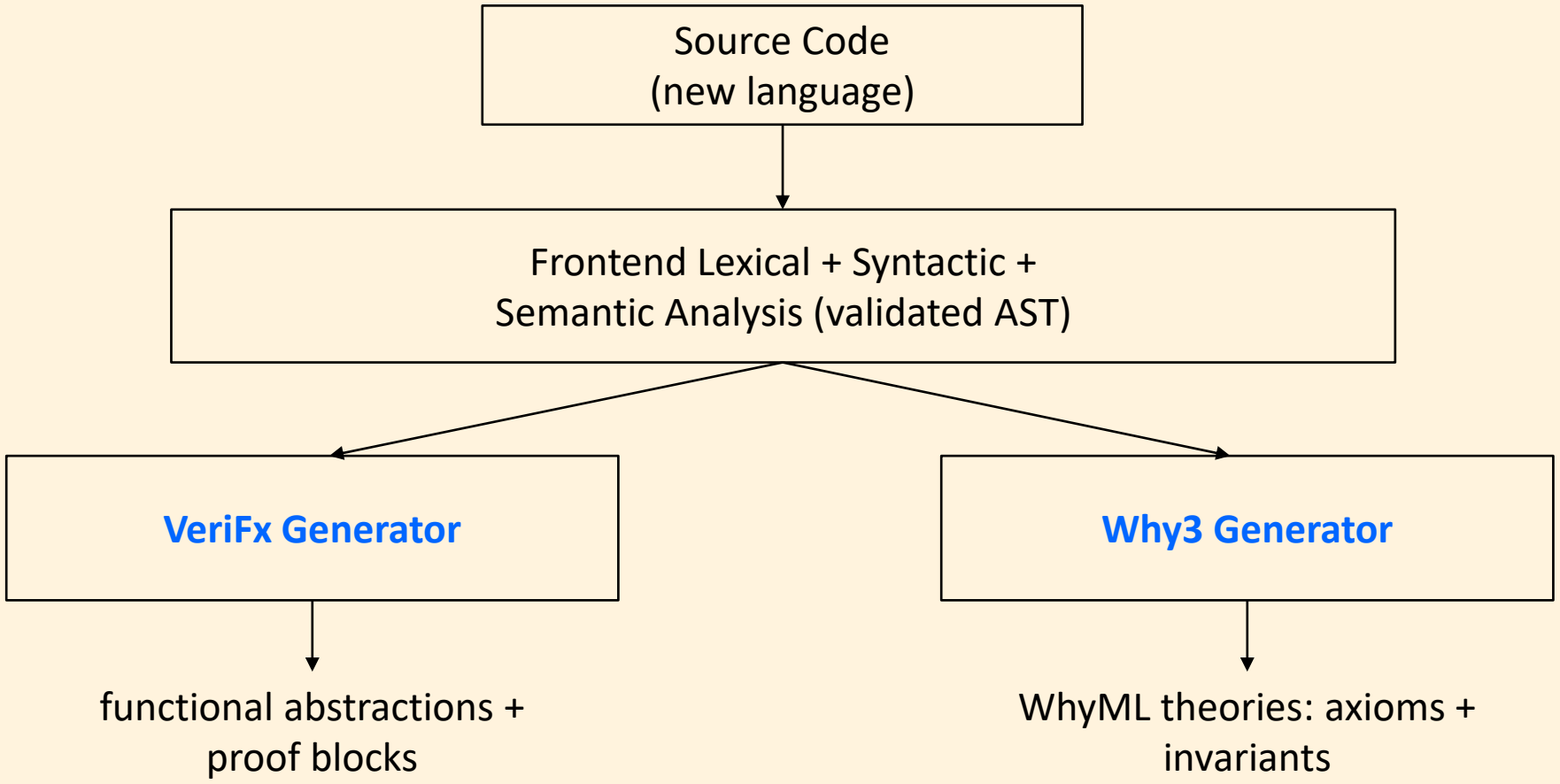
- Partial Order (≤)
- Least Upper Bound (LUB)



Architecture and Compilation Pipeline - Overview



Architecture and Compilation Pipeline - Overview



Specification Language - Modules

```
module RGA : CmRDT
  . . .
  type elem = { atom: atom_type, timestamp: lamport_clock }
  type payload =
    { clock: lamport_clock,
      added: set elem,
      removed: set elem,
      edges: map<elem, elem>,
      bottom_value: atom_type } [@vfx OpTwoPSet: vertices]
  . . .
  type operation = Add: { l w: elem } | Rem: elem
  . . .
  val execute (op: operation, a: payload) : payload = . . .

  val compare (a b: payload) : boolean =
    set.subset(a.added, b.added) && set.subset(a.removed, b.removed)

  val equals (a b: payload) : boolean =
    compare(a, b) && compare(b, a)
  . . .
end
```

Specification Language - Interfaces and Axioms

```
interface CvRDT[@proof]
  type payload

  val create () : payload
  val merge (a b: payload) : payload
  val compare (a b: payload) : boolean
  val equals (a b: payload) : boolean

  axiom commutative(merge)
  axiom idempotent(merge)
  axiom associative(merge)
  axiom equivalence(compare)
end
```

```
interface CmRDT[@proof]
  type payload
  type operation

  val create () : payload
  val execute (op: operation, state: payload) : payload
  val compare (a b: payload) : boolean
  val equals (a b: payload) : boolean

  axiom op_commutative(execute)
end
```

Compiling RGA - Why3

```
// specification
val execute (op: operation, a: payload)
    : payload =
  match op with
  | Add left e ->
      find_insert_position(e.timestamp, e,
        left, successor(left, a), a,
        set.cardinal(a.added) + 1)
  | Rem e ->
      { clock: a.clock,
        added: a.added,
        removed: set.add(e, a.removed),
        edges: a.edges,
        bottom_value: a.bottom_value }
  end
```

```
(* generated for Why3 *)
module RGA : CmRDT
  . . .
  type t = { payload: payload; }
  . . .
  function execute (op: operation) (a: t) : t
    = match op with
      | Add left e ->
          find_insert_position e.timestamp e left
            (successor left a) a
            (Fset.cardinal a.payload.added + 1)
      | Rem e ->
          { payload = { clock = a.payload.clock;
              added = a.payload.added;
              removed = Fset.add e a.payload.removed;
              edges = a.payload.edges;
              bottom_value = a.payload.bottom_value } }
      end
end
```

Compiling RGA - VeriFx Composition

```
// specification
type lamport_clock =
  { replica: integer,
    counter: integer }
    [@vfx class : smaller_clock, increment_clock,
        sync, smaller_or_equal, bigger]

type payload =
  { clock: lamport_clock,
    added: set elem,
    removed: set elem,
    edges: map<elem, elem>,
    bottom_value: atom_type } [@vfx OpTwoPSet: vertices]
```

```
// generated for VeriFx
class RGA[V](clock: LamportClock,
    vertices: OpTwoPSet[Elem[V]],
    edges: Map[Elem[V], Elem[V]],
    bottom_value: V)
  extends CmRDT[RGAOp[V], RGAMsg[V], RGA[V]] {
  . . .
}

object RGA
  extends CmRDTProof1[RGAOp, RGAMsg, RGA]
```

Compiling RGA - Proving Termination

```
// specification
val find_insert_position (stamp: lamport_clock,
      e left right: elem, a: payload,
      fuel: integer) : payload
  variant (fuel)
  =
    if fuel <= 0 then
      . . .
    else if smaller_clock(stamp,
                          right.timestamp) then
      find_insert_position(stamp, e, right,
          successor(right, a), a, fuel - 1)
    else
      . . .
```

```
// generated for VeriFx
private def find_insert_position(
    stamp: LamportClock, e: Elem[V],
    left: Elem[V], right: Elem[V]): RGA[V] = {
  if (stamp.smallerClock(right.timestamp)) {
    this.find_insert_position(stamp, e, right,
        this.successor(right))
  } else {
    . . .
    new RGA(newClock, this.vertices.add(e),
        this.edges.add(e, right).add(left, e),
        this.bottom_value)
  }
}
```

From Axioms to Proof Obligations

```
// specification
interface CmRDT[@proof]
  . . .
  axiom op_commutative(execute)
end
```

```
(* generated for Why3 *)
module CmRDT
  . . .
  axiom op_commutative:
    forall o1 o2: operation, a: t.
      equals (execute o2 (execute o1 a))
             (execute o1 (execute o2 a))
end
```

```
// generated for VeriFx
trait CmRDTProof1[. . .] {
  proof is_a_CmRDT[S] {
    forall (s1: T[S], s2: T[S], s3: T[S],
            x: Op[S], y: Op[S]) {
      val msg1 = s1.prepare(x)
      val msg2 = s2.prepare(y)
      ( s1.reachable() && . . . ) =>: {
        s3.tryEffect(msg1).tryEffect(msg2).equals(
          s3.tryEffect(msg2).tryEffect(msg1)) && . . .
      }
    }
  }
}
```

Case Studies - Specification Effort

CRDT	Model	LoC Written	LoC (VeriFx)	LoC (WhyML)
GCounter_CRDT	State	13	24	35
PNCounter_CRDT	State	18	23	25
VGCounter	State	26	27	51
OpCounter_CRDT	Operation	14	17	19
NestedMap	State	29	28	57
GSet	State	15	9	37
TwoPSet	State	17	16	43
OpTwoPSet	Operation	24	23	22
UWMap	State	45	20	58
RGA	Operation	196	182	119

Case Studies - Verification Results

CRDT	VeriFx	Why3
Other 6 CRDTs	Automatic	Automatic
NestedMap, UWMap	Automatic	Automatic, auxiliary lemma
PNCounter_CRDT	Automatic	One manual step
RGA	**Timeout**	**Partial**

RGA Proof - Induction in Why3

```
lemma find_insert_position_facts
    (stamp: lamport_clock, e left right: elem,
     a: payload, fuel: integer)
  variant (fuel)
  ensures { (find_insert_position(stamp, e, left,
              right, a, fuel)).added ==
            set.add(e, a.added) }
  ensures { (find_insert_position(stamp, e, left,
              right, a, fuel)).removed ==
            a.removed }
= if fuel <= 0 then true
  else if smaller_clock(stamp, right.timestamp) then
    find_insert_position_facts(stamp, e, right,
        successor(right, a), a, fuel - 1)
  else true
```

RGA Proof - Operation Pairs

Operation pair	Same elements (equals)	Same order (equals_full)
`(Rem, Rem)`	Proved	Proved
`(Add, Rem)`	Proved	Proved
`(Rem, Add)`	Proved	Proved
`(Add, Add)`	Proved	**Future work**

Conclusion

- Research Question 1:
  - Yes, one specification language expresses both state-based and operation-based CRDTs, and is compiled automatically to VeriFx and Why3.

Conclusion

- Research Question 1:
  - Yes, one specification language expresses both state-based and operation-based CRDTs, and is compiled automatically to VeriFx and Why3.
- Research Question 2:
  - Yes, for every CRDT the specification is shorter than the VeriFx and WhyML code combined.

Conclusion

- Research Question 1:
  - Yes, one specification language expresses both state-based and operation-based CRDTs, and is compiled automatically to VeriFx and Why3.
- Research Question 2:
  - Yes, for every CRDT the specification is shorter than the VeriFx and WhyML code combined.
- Research Question 3:
  - Both tools agree on nine of the ten CRDTs, and Why3 goes further on RGA, where VeriFx times out.

Future Work - Proofs

- Complete the RGA Proof:
  - Prove that two insertions also leave the elements in the same order.

Future Work - Proofs

- Complete the RGA Proof:
  - Prove that two insertions also leave the elements in the same order.
- Share Results Across Backends:
  - Let a property proved in one backend become an assumption in the other.
  - VeriFx could rely on the three RGA pairs that Why3 already proves.

Future Work - Framework

- Broader Evaluation:
  - Further recursive or unbounded CRDTs.

Future Work - Framework

- Broader Evaluation:
  - Further recursive or unbounded CRDTs.
- Stronger Type Checking:
  - Match record literals by the types of their fields, not only by their names.

An Integrated Framework for the Development of CRDTs

A Deductive Study on Verification of Replicated Data Types

João Garção

Advisor: Mário Pereira

Co-Advisor: Carla Ferreira

November 20, 2026